

가설검정

```
In [3]: import numpy as np
import pandas as pd
# 정규분포, t분포, 신뢰구간(z분포, t분포), 가설검정(z-test, t-test, paired t-test)
import scipy.stats as stats
# 비율의 신뢰구간, 비율의 가설검정, two-sample ttest, 평균차의 신뢰구간
import statsmodels.stats.api as sms

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

평균에 대한 검정

우측검정 예시

```
In [4]: data1 = pd.read_csv("department.csv")
data1
```

Out[4]:

	Accounts
0	116.65
1	87.09
2	328.01
3	75.73
4	211.82
...	...
395	162.00
396	337.28
397	230.45
398	237.37
399	219.35

400 rows × 1 columns

```
In [5]: data1['Accounts'].mean()
```

Out[5]: 177.99650000000003

```
In [6]: '''
`scipy.stats` 패키지의 `norm.cdf()` 함수는 정규 분포의
누적 분포 함수(Cumulative Distribution Function, CDF)를 계산하는 함수입니다.
정규 분포의 누적 분포 함수는 주어진 값까지의 확률을 나타냅니다.

특징:
- `norm.cdf()` 함수는 정규 분포를 따르는 확률 변수가
  특정 값보다 작거나 같은 확률을 반환합니다.
- 이 함수는 표준 정규 분포(평균=0, 표준편차=1)에 대해서만 작동하므로,
  다른 평균과 표준편차를 갖는 정규 분포의 경우에는
  데이터를 표준화해야 합니다.
```

인자:

- **x**: 누적 분포 함수를 계산할 값 또는 배열입니다.
정규 분포에서 해당 값까지의 누적 확률을 계산합니다.
- **loc**: 정규 분포의 평균을 나타내는 매개변수입니다. 기본값은 0입니다.
- **scale**: 정규 분포의 표준편차를 나타내는 매개변수입니다.
기본값은 1입니다.

리턴값:

- 입력된 값에 대한 정규 분포의 누적 확률을 반환합니다.
따라서 반환값은 0과 1 사이의 값을 가집니다.

사용법:

```
from scipy.stats import norm
```

```
# 평균이 0이고 표준편차가 1인 정규 분포에서 x=2까지의 누적 확률 계산
probability = norm.cdf(2)
```

```
# 평균이 5이고 표준편차가 2인 정규 분포에서 x=6까지의 누적 확률 계산
probability = norm.cdf(6, loc=5, scale=2)
```

```
`norm.cdf()` 함수는 주어진 값까지의 누적 확률을 계산하여 반환합니다.  
평균과 표준편차를 지정하여 다른 정규 분포의 경우에도 활용할 수 있습니다.  
'''
```

```
xbar = data1['Accounts'].mean()
sigma = 65
n = len(data1)
```

```
# 가설검정(one sample z-test)
1-stats.norm.cdf(xbar, loc=170, scale=sigma/np.sqrt(n))
```

Out[6]: 0.0069379221924118495

In [7]: xbar

Out[7]: 177.99650000000003

In [8]: '''
`scipy.stats` 패키지의 `norm.ppf()` 함수는 정규 분포의

누적 분포 함수(Cumulative Distribution Function, CDF)의 역함수(Inverse Function)로, 주어진 누적 확률에 해당하는 값을 찾는 함수입니다. 이 함수를 사용하여 특정 확률에 해당하는 값(퍼센트 포인트)을 계산할 수 있습니다.

특징:

- `norm.ppf()` 함수는 주어진 확률에 해당하는 값(퍼센트 포인트)을 계산합니다.
- 이 함수는 표준 정규 분포(평균=0, 표준편차=1)에 대해서만 작동하므로, 다른 평균과 표준편차를 갖는 정규 분포의 경우에는 데이터를 표준화해야 합니다.

인자:

- `q`: 계산할 퍼센트 포인트에 해당하는 누적 확률 값입니다. 이 값은 0과 1 사이의 실수이어야 합니다.
- `loc`: 정규 분포의 평균을 나타내는 매개변수입니다. 기본값은 0입니다.
- `scale`: 정규 분포의 표준편차를 나타내는 매개변수입니다. 기본값은 1입니다.

리턴값:

- 입력된 누적 확률 값에 해당하는 퍼센트 포인트를 반환합니다.

사용법:

```
from scipy.stats import norm
```

```
# 표준 정규 분포에서 누적 확률이 0.95에 해당하는 퍼센트 포인트 계산
value = norm.ppf(0.95)
```

```
# 평균이 5이고 표준편차가 2인 정규 분포에서
# 누적 확률이 0.8에 해당하는 퍼센트 포인트 계산
value = norm.ppf(0.8, loc=5, scale=2)
```

```
`norm.ppf()` 함수를 사용하여
주어진 누적 확률에 해당하는 값을 계산할 수 있습니다.
평균과 표준편차를 지정하여 다른 정규 분포의 경우에도
활용할 수 있습니다.
'''
```

```
stats.norm.ppf(0.95, loc=170, scale=sigma/np.sqrt(n))
```

Out[8]: 175.3457742875923

```
In [9]: data1['Accounts'].std()
```

```
Out[9]: 68.36697563255856
```

```
In [10]: 1-stats.t.cdf(2.340, df=n-1)
```

```
Out[10]: 0.00988797117245388
```

```
In [11]: stats.t.ppf(0.95, df=n-1)
```

```
Out[11]: 1.6486815335554057
```

```
In [12]: '''
`scipy.stats` 패키지의 `ttest_1samp()` 함수는 한 집단의 평균에 대한
t-검정(t-test)을 수행하는 함수입니다.
이 함수를 사용하여 하나의 샘플 집단의 평균이
기준값(예를 들어, 모집단의 평균)과 같은지를 검정할 수 있습니다.

특징:
- `ttest_1samp()` 함수는 하나의 샘플 집단에 대한 t-검정을 제공합니다.
- 이 함수는 평균의 차이가 통계적으로 유의미한지를 검정합니다.
- t-검정은 표본의 평균과 모집단의 평균 간의 차이를 검정하는 데 사용됩니다.

인자:
- a: 검정할 데이터 샘플의 배열 또는 리스트입니다.
- popmean: 검정하려는 모집단의 평균입니다.
    만약 None으로 지정하면 평균이 0인 것으로 간주됩니다.
- axis: 검정할 축을 지정합니다. 기본값은 None으로,
    입력된 배열 전체에 대해 검정을 수행합니다.
- nan_policy: NaN 값의 처리 방법을 지정합니다.
    "propagate", "raise", "omit" 중 하나를 선택할 수 있습니다.
    기본값은 "propagate"입니다.

리턴값:
- t-statistic: t-검정 통계량 값입니다.
- p-value: t-검정 결과에 대한 p-value 값입니다.
    유의수준과 비교하여 검정 결과를 해석합니다.

사용법:
```

```

from scipy.stats import ttest_1samp

# 데이터 샘플과 모집단의 평균을 비교하여 t-검정 수행
t_statistic, p_value = ttest_1samp(data, popmean=population_mean)

# NaN 값이 포함된 데이터에서 t-검정 수행 시 NaN 처리 방법 지정
t_statistic, p_value =
    ttest_1samp(data, popmean=population_mean, nan_policy='omit')

`ttest_1samp()` 함수를 사용하여 주어진 데이터 샘플의 평균이
모집단의 평균과 통계적으로 유의미한 차이가 있는지를
검정할 수 있습니다. 이를 통해 데이터가 특정 모집단에서 얻어졌는지를
판단하거나, 실험 결과의 유효성을 검증할 수 있습니다.
'''

stats.ttest_1samp(data1, popmean=170, alternative='greater')

```

Out[12]: TtestResult(statistic=array([2.33928733]), pvalue=array([0.00990659]), df=array([399]))

좌측검정 예시

```

In [13]: data2 = pd.read_csv("ketchup.csv")
data2

```

Out[13]:

Ketchup

0	15.8
1	16.0
2	16.2
3	15.7
4	15.4
5	16.1
6	16.2
7	17.3
8	15.0
9	16.8
10	15.6
11	15.9
12	16.0
13	16.2
14	15.6
15	16.0
16	16.8
17	15.7
18	15.6
19	15.3
20	15.7
21	15.8

Ketchup	
22	15.6
23	15.5
24	15.7

```
In [14]: xbar = data2['Ketchup'].mean()
s = data2['Ketchup'].std()
n = len(data2)
mu0 = 16
print('Sample mean: ', xbar)
print('Sample St.Dev: ', s)
```

Sample mean: 15.899999999999999
Sample St.Dev: 0.5016638981097472

```
In [15]: t_value = (xbar-mu0) / (s/np.sqrt(n))
t_value
```

Out[15]: -0.996683241277657

```
In [16]: stats.t.cdf(-0.997, df=n-1)
```

Out[16]: 0.16435555565598453

```
In [17]: stats.t.ppf(0.05, df=n-1)
```

Out[17]: -1.7108820799094282

```
In [18]: stats.ttest_1samp(data2, popmean=16, alternative='less')
```

Out[18]: TtestResult(statistic=array([-0.99668324]), pvalue=array([0.16443087]), df=array([24]))

양측검정 예시


```
In [19]: data3 = pd.read_csv("assembly_time.csv")
data3
```

```
Out[19]:
```

	Time
0	145
1	128
2	153
3	141
4	119
...	...
95	140
96	134
97	157
98	103
99	106

100 rows × 1 columns

```
In [20]: xbar = data3['Time'].mean()
s = data3['Time'].std()
n = len(data3)
mu0 = 130
print('Sample mean: ', xbar)
print('Sample St.Dev: ', s)
```

Sample mean: 126.8

Sample St.Dev: 14.482312444405203

```
In [21]: t_value = (xbar - mu0) / (s / np.sqrt(n))
t_value
```

```
Out[21]: -2.2095918813270905
```

```
In [22]: 2*stats.t.cdf(-2.210, df=n-1)
```

```
Out[22]: 0.02940876862806306
```

```
In [23]: stats.t.ppf(0.025, df=n-1)
```

```
Out[23]: -1.9842169515086832
```

```
In [24]: stats.ttest_1samp(data3, popmean=130, alternative='less')
```

```
Out[24]: TtestResult(statistic=array([-2.20959188]), pvalue=array([0.01471901]), df=array([99]))
```

비율에 대한 검정

```
In [25]: data4 = pd.read_csv("election.csv")  
data4
```

Out[25]:

Votes	
0	2
1	1
2	1
3	2
4	2
...	...
760	2
761	2
762	1
763	2
764	2

765 rows × 1 columns

In [26]: `data4.value_counts()`

Out[26]: Votes
 2 407
 1 358
 dtype: int64

In [27]: `n = len(data4)`
`p_hat = data4.value_counts()[2]/n`
`p_hat`

Out[27]: 0.5320261437908497

In [28]: `p0 = 0.5`
`Z_value = (p_hat-p0) / np.sqrt(p0*(1-p0)/n)`

Z_value

Out[28]: 1.7715987388523589

In [29]: `1-stats.norm.cdf(Z_value, loc=0, scale=1)`

Out[29]: 0.03823059460087119

In [30]: `1-stats.norm.cdf(0.532, loc=0.5, scale=np.sqrt(0.5*0.5/765))`

Out[30]: 0.03835086688510403

In [31]: `stats.norm.ppf(0.95, loc=0, scale=1)`

Out[31]: 1.6448536269514722

In [32]: '''
``statsmodels.stats.api`` 패키지의 ``proportions_ztest()`` 함수는
 두 범주형 변수 간의 비율 차이에 대한 z-검정(z-test)을 수행하는 함수입니다.
 이 함수를 사용하여 두 개의 범주형 변수 간의 비율이
 통계적으로 유의미한 차이가 있는지를 검정할 수 있습니다.

특징:

- ``proportions_ztest()`` 함수는 두 개의 범주형 변수 간의 비율 차이를 검정하는 데 사용됩니다.
- 이 함수는 두 비율 간의 차이가 통계적으로 유의미한지를 결정하는 z-검정 통계량을 제공합니다.

인자:

- **count**: 각 범주에서의 성공(예: 성공한 시행의 수, 특정 조건을 만족하는 경우의 수)을 나타내는 배열 또는 리스트입니다.
- **nobs**: 각 범주에서의 총 시행 수(예: 총 시도 횟수)를 나타내는 배열 또는 리스트입니다.
- **alternative**: 대립가설을 지정하는 문자열입니다.
 "two-sided", "larger", "smaller" 중 하나를 선택할 수 있습니다.
 기본값은 "two-sided"로, 두 비율이 같지 않다는 것을 검정합니다.
- **prop_var**: 두 비율의 표본 분산이 같은지 여부를 나타내는 boolean 값입니다.
 기본값은 True로, 두 표본의 분산이 같다고 가정합니다.
 False로 설정하면 Welch's t-test를 수행합니다.

- **correction**: 이항 분포의 정규 근사에서 사용되는 보정 방법을 나타내는 문자열입니다.
"auto", "binomial", "approximate" 중 하나를 선택할 수 있습니다.
기본값은 "auto"로, 샘플 크기가 크면 정규 근사를 사용합니다.
- **return_results**: 결과를 튜플로 반환할지 여부를 나타내는 **boolean** 값입니다.
기본값은 **False**로, z-검정 통계량과 **p-value**만 반환합니다.

리턴값:

- **z-statistic**: z-검정 통계량 값입니다.
두 비율의 차이가 통계적으로 유의미한지를 판단하는 데 사용됩니다.
- **p-value**: z-검정 결과에 대한 **p-value** 값입니다.
유의수준과 비교하여 검정 결과를 해석합니다.

사용법:

```
from statsmodels.stats.api import proportions_ztest

# 두 비율 간의 z-검정 수행
z_statistic, p_value = proportions_ztest(count=successes, nobs=total_trials)

# 대립가설이 "larger"일 때 두 비율 간의 z-검정 수행
z_statistic, p_value = proportions_ztest(count=successes, nobs=total_trials,
                                          alternative='larger')

`proportions_ztest()` 함수를 사용하여 두 개의 범주형 변수 간의
비율 차이를 검정할 수 있습니다.
이를 통해 두 집단 간의 비율이 통계적으로
유의미한 차이가 있는지를 검정할 수 있습니다.
'''

sms.proportions_ztest(count=data4.value_counts()[2], nobs=n,
                      value=0.5, prop_var=0.5, alternative='larger')
```

Out[32]: (1.7715987388523589, 0.03823059460087115)

분산에 대한 검정

```
In [33]: popcorn = pd.DataFrame([198, 201, 199, 189, 200,
                                199, 198, 189, 205, 195])
s = popcorn.std(); print(s**2)
```

```
0    25.566667
dtype: float64
```

```
In [92]: '''
`scipy.stats` 패키지의 `chi2.cdf()` 함수는
카이제곱 분포(Chi-square distribution)의
누적 분포 함수(Cumulative Distribution Function, CDF)를
계산하는 함수입니다.
카이제곱 분포는 주로 독립성 검정, 적합도 검정 등에서 사용되며,
누적 분포 함수는 주어진 값까지의 누적 확률을 나타냅니다.

특징:
- `chi2.cdf()` 함수는 카이제곱 분포의 누적 분포 함수를 계산합니다.
- 이 함수는 주어진 값까지의 누적 확률을 반환합니다.

인자:
- x: 누적 분포 함수를 계산할 값 또는 배열입니다.
- df: 자유도(degree of freedom)를 나타내는 매개변수입니다.
    카이제곱 분포의 형태를 결정합니다.
- loc: 분포의 위치를 나타내는 매개변수입니다. 기본값은 0입니다.
- scale: 분포의 스케일을 나타내는 매개변수입니다. 기본값은 1입니다.

리턴값:
- 입력된 값에 대한 카이제곱 분포의 누적 확률을 반환합니다.
    따라서 반환값은 0과 1 사이의 값을 가집니다.

사용법:
from scipy.stats import chi2

# 자유도가 3인 카이제곱 분포에서 x=5까지의 누적 확률 계산
probability = chi2.cdf(5, df=3)

# 자유도가 10인 카이제곱 분포에서 x=20까지의 누적 확률 계산
probability = chi2.cdf(20, df=10)

`chi2.cdf()` 함수를 사용하여 주어진 값까지의 누적 확률을 계산할 수 있습니다.
```

자유도를 조정하여 다른 형태의 카이제곱 분포에 대해서도
누적 확률을 계산할 수 있습니다.

```
'''
```

```
'''
```

`scipy.stats` 패키지의 `chi2.ppf()` 함수는 카이제곱 분포의
누적 분포 함수의 역함수(Inverse Function)로,
주어진 누적 확률에 해당하는 값을 찾는 함수입니다.

특징:

- `chi2.ppf()` 함수는 주어진 누적 확률에 해당하는 카이제곱 분포의
값을 계산합니다.
- 이 함수는 카이제곱 분포의 퍼센트 포인트(Percent Point Function, PPF)를
계산하여 제공합니다.

인자:

- **q**: 계산할 퍼센트 포인트에 해당하는 누적 확률 값입니다.
이 값은 0과 1 사이의 실수이어야 합니다.
- **df**: 자유도(degree of freedom)를 나타내는 매개변수입니다.
카이제곱 분포의 형태를 결정합니다.
- **loc**: 분포의 위치를 나타내는 매개변수입니다. 기본값은 0입니다.
- **scale**: 분포의 스케일을 나타내는 매개변수입니다. 기본값은 1입니다.

리턴값:

- 입력된 누적 확률 값에 해당하는 카이제곱 분포의 값을 반환합니다.

사용법:

```
from scipy.stats import chi2
```

```
# 자유도가 3인 카이제곱 분포에서 누적 확률이 0.95에 해당하는 퍼센트 포인트 계산
value = chi2.ppf(0.95, df=3)
```

```
# 자유도가 10인 카이제곱 분포에서 누적 확률이 0.05에 해당하는 퍼센트 포인트 계산
value = chi2.ppf(0.05, df=10)
```

`chi2.ppf()` 함수를 사용하여 주어진 누적 확률에 해당하는
카이제곱 분포의 값을 계산할 수 있습니다.

자유도를 조정하여 다른 형태의 카이제곱 분포에 대해서도 값을 계산할 수 있습니다.

```
'''
```

```
n = len(popcorn)
```

```
df=n-1
```

```
test_stat=(n-1)* s**2/25
print("n=", n)
print("test stat=", test_stat.values)
print("P-value=",1-stats.chi2.cdf(test_stat, df))
print("chi-square critical value =", stats.chi2.ppf(0.95, df))
```

n= 10

test stat= [9.204]

P-value= [0.41866044]

chi-square critical value = 16.918977604620448

두 모집단 평균차이에 대한 검정

등분산

```
In [39]: data5 = pd.read_csv("chair.csv")
data5
```


Out[39]:

	Method A	Method B
0	6.8	5.2
1	5.0	6.7
2	7.9	5.7
3	5.2	6.6
4	7.6	8.5
5	5.0	6.5
6	5.9	5.9
7	5.2	6.7
8	6.5	6.6
9	7.4	4.2
10	6.1	4.2
11	6.2	4.5
12	7.1	5.3
13	4.6	7.9
14	6.0	7.0
15	7.1	5.9
16	6.1	7.1
17	5.0	5.8
18	6.3	7.0
19	7.0	5.7
20	6.4	5.9
21	6.1	4.9

	Method A	Method B
22	6.6	5.3
23	7.7	4.2
24	6.4	7.1

```
In [40]: data5.mean()
```

```
Out[40]: Method A    6.288
Method B    6.016
dtype: float64
```

```
In [41]: data5.var()
```

```
Out[41]: Method A    0.847767
Method B    1.303067
dtype: float64
```

```
In [42]: 2*(1-stats.t.cdf(0.9273, df=25+25-2))
```

```
Out[42]: 0.35841023759624013
```

```
In [43]: stats.t.ppf(0.975, df=25+25-2)
```

```
Out[43]: 2.0106347546964454
```

```
In [44]: '''
`scipy.stats` 패키지의 `ttest_ind()` 함수는 두 독립적인 집단 간의
평균 차이에 대한 t-검정(t-test)을 수행하는 함수입니다.
이 함수를 사용하여 두 집단의 평균이 통계적으로
유의미한 차이가 있는지를 검정할 수 있습니다.

특징:
- `ttest_ind()` 함수는 두 개의 독립적인 집단 간의 평균 차이를 검정합니다.
- 이 함수는 평균의 차이가 통계적으로 유의미한지를 결정하는
  t-검정 통계량과 p-value를 제공합니다.
```

인자:

- **a, b**: 비교할 두 개의 독립적인 데이터 샘플입니다.
각각의 데이터 샘플은 배열, 리스트 또는 **pandas Series** 형태일 수 있습니다.
- **equal_var**: 두 집단의 분산이 같은지 여부를 지정하는 **boolean** 값입니다.
기본값은 **True**로, 두 집단의 분산이 같다고 가정합니다.
False로 설정하면 **Welch's t-test**를 수행합니다.
- **nan_policy**: NaN 값의 처리 방법을 지정합니다.
"propagate", "raise", "omit" 중 하나를 선택할 수 있습니다.
기본값은 "propagate"입니다.

리턴값:

- **t-statistic**: t-검정 통계량 값입니다.
두 집단 간의 평균 차이가 통계적으로 유의미한지를 판단하는 데 사용됩니다.
- **p-value**: t-검정 결과에 대한 **p-value** 값입니다.
유의수준과 비교하여 검정 결과를 해석합니다.

사용법:

```
from scipy.stats import ttest_ind
```

두 집단 간의 t-검정 수행

```
t_statistic, p_value = ttest_ind(group1_data, group2_data)
```

두 집단 간의 분산이 같지 않은 경우(등분산성 가정이 위배된 경우)

```
t_statistic, p_value =  
    ttest_ind(group1_data, group2_data, equal_var=False)
```

`ttest\_ind()` 함수를 사용하여 두 독립적인 집단 간의
평균 차이를 검정할 수 있습니다.

이를 통해 두 집단 간의 특정 변수의 차이가 통계적으로
유의미한지를 검정할 수 있습니다.

```
'''
```

```
stats.ttest_ind(data5['Method A'], data5['Method B'],  
                equal_var=True, alternative='two-sided')
```

```
Out[44]: Ttest_indResult(statistic=0.9273326034699476, pvalue=0.3583934877441609)
```

이분산

```
In [45]: data6 = pd.read_csv("cereal.csv")
data6
```

```
Out[45]:
```

	Consumers	Nonconsumers
0	568.0	705
1	498.0	819
2	589.0	706
3	681.0	509
4	540.0	613
...	...	...
102	NaN	594
103	NaN	596
104	NaN	750
105	NaN	582
106	NaN	788

107 rows × 2 columns

```
In [46]: data6.mean()
```

```
Out[46]: Consumers      604.023256
Nonconsumers    633.233645
dtype: float64
```

```
In [47]: data6.var()
```

```
Out[47]: Consumers      4102.975637
Nonconsumers    10669.765650
dtype: float64
```

```
In [48]: stats.t.cdf(-2.091, df=122.6)
```

```
Out[48]: 0.01929656032126123
```

```
In [49]: stats.t.cdf(-2.091, df=123)
```

```
Out[49]: 0.019293184097103244
```

```
In [50]: stats.t.ppf(0.05, df=123)
```

```
Out[50]: -1.6573363970416897
```

```
In [52]: stats.ttest_ind(data6['Consumers'].dropna(), data6['Nonconsumers'],
                        equal_var=False, alternative='less')
```

```
Out[52]: Ttest_indResult(statistic=-2.0910731586290563, pvalue=0.019293211531347457)
```

등분산 검정

```
In [53]: f = data5['Method A'].var()/data5['Method B'].var()
f
```

```
Out[53]: 0.6505934718100891
```

```
In [54]: df1 = data5['Method A'].count()-1
df2 = data5['Method B'].count()-1
stats.f.cdf(f, df1, df2)
```

```
Out[54]: 0.14960564238494933
```

```
In [55]: f = data6['Consumers'].var()/data6['Nonconsumers'].var()
f
```

```
Out[55]: 0.38454224501587925
```

```
In [56]: '''
`scipy.stats` 패키지의 `f.cdf()` 함수는 F 분포(F-distribution)의
```

누적 분포 함수(Cumulative Distribution Function, CDF)를 계산하는 함수입니다.
F 분포는 일반적으로 분산 비율을 비교하는 분석에 사용되며,
누적 분포 함수는 주어진 값까지의 누적 확률을 나타냅니다.

특징:

- `f.cdf()` 함수는 F 분포의 누적 분포 함수를 계산합니다.
- 이 함수는 주어진 값까지의 누적 확률을 반환합니다.

인자:

- `x`: 누적 분포 함수를 계산할 값 또는 배열입니다.
- `dfn`: 분자 자유도(degree of freedom)를 나타내는 매개변수입니다.
분자의 자유도는 그룹의 수에 따라 달라집니다.
- `dfd`: 분모 자유도(degree of freedom)를 나타내는 매개변수입니다.
분모의 자유도는 샘플의 크기에 따라 달라집니다.
- `loc`: 분포의 위치를 나타내는 매개변수입니다. 기본값은 0입니다.
- `scale`: 분포의 스케일을 나타내는 매개변수입니다. 기본값은 1입니다.

리턴값:

- 입력된 값에 대한 F 분포의 누적 확률을 반환합니다.
반환값은 0과 1 사이의 값을 가집니다.

사용법:

```
from scipy.stats import f
```

```
# 분자 자유도가 3이고 분모 자유도가 5인 F 분포에서
# x=2까지의 누적 확률 계산
probability = f.cdf(2, dfn=3, dfd=5)
```

```
# 분자 자유도가 10이고 분모 자유도가 8인 F 분포에서
# x=1.5까지의 누적 확률 계산
probability = f.cdf(1.5, dfn=10, dfd=8)
```

```
`f.cdf()` 함수를 사용하여 주어진 값까지의 누적 확률을 계산할 수 있습니다.
분자와 분모의 자유도를 조정하여
다른 형태의 F 분포에 대해서도 누적 확률을 계산할 수 있습니다.
'''
```

```
df1 = data6['Consumers'].count()-1
df2 = data6['Nonconsumers'].count()-1
stats.f.cdf(f, df1, df2)
```

Out[56]: 0.0003684331512590764

정규성 검정

Q-Q plot

```
In [57]: import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [58]: iris = pd.read_csv("iris.csv")
```

```
In [95]: '''
`statsmodels.api` 패키지의 `qqplot()` 함수는
QQ 플롯을 생성하는 함수입니다.
QQ 플롯은 주어진 데이터가 정규 분포를 따르는지를 시각적으로 확인하는 데 사용됩니다.
이 그래프는 데이터의 분위수와 정규 분포의 분위수를 비교하여
데이터가 정규 분포에 얼마나 잘 부합하는지를 확인할 수 있습니다.
```

특징:

- `qqplot()` 함수는 주어진 데이터가 정규 분포를 따르는지를 시각적으로 확인하는 QQ 플롯을 생성합니다.
- 이 함수는 주어진 데이터의 샘플 분위수와 정규 분포의 이론적 분위수를 비교하여 정규성을 평가합니다.

인자:

- **data**: QQ 플롯을 생성할 데이터 배열, 리스트 또는 pandas Series입니다.
- **line**: QQ 플롯에 표시할 직선의 종류를 지정하는 문자열입니다.
"45"는 45도 기울기의 대각선 직선을 나타내며,
"s"는 표준화된 데이터의 기대값을 나타내는 직선을 나타냅니다.
기본값은 "45"입니다.
- **ax**: 그래프를 표시할 matplotlib 축(ax) 객체입니다.
이 인자를 사용하여 동일한 figure에 여러 그래프를 그릴 수 있습니다.
- **fit**: QQ 플롯에 로지스틱 함수 또는 분위수-분위수 회귀선을 추가할지 여부를 지정하는 부울(boolean) 값입니다. 기본값은 False입니다.
- **line_kwarg**: 직선에 대한 추가적인 속성을 설정하는 딕셔너리입니다.

사용법:

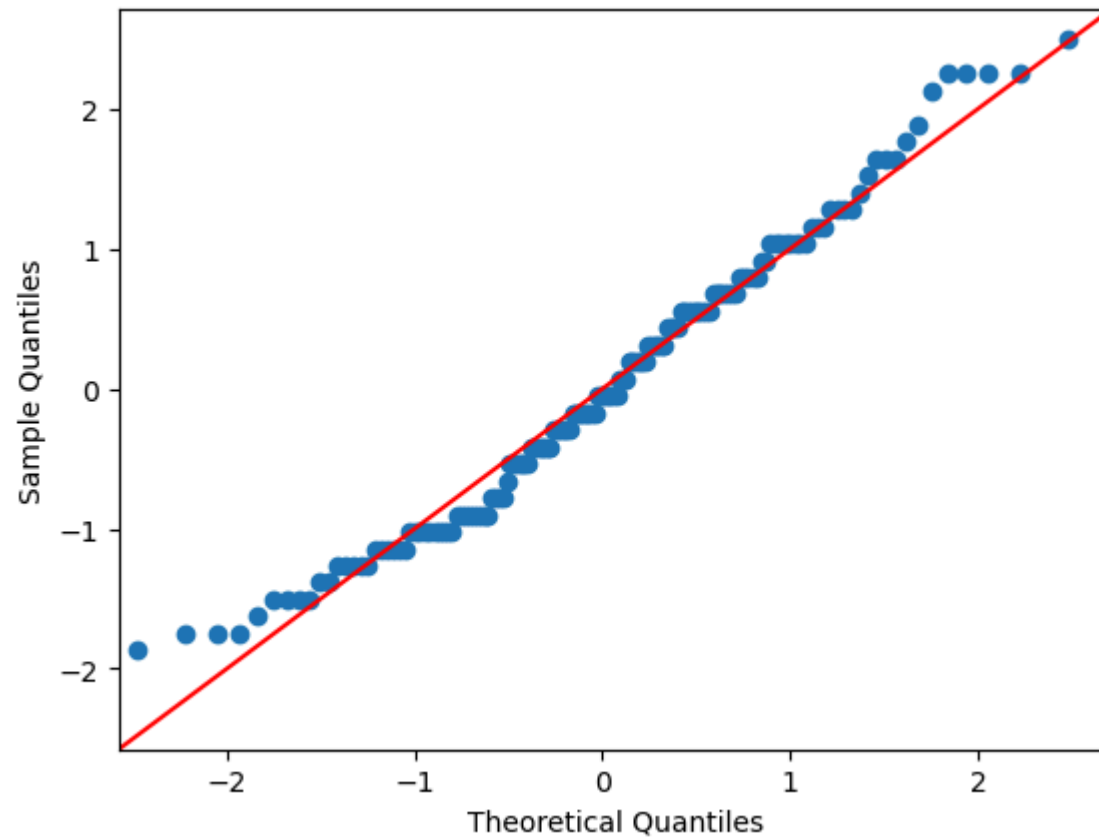
```
import statsmodels.api as sm
import matplotlib.pyplot as plt
```

```
# 데이터 생성 예시 (정규 분포에서 생성된 샘플)
data = np.random.normal(loc=0, scale=1, size=1000)
```

```
# QQ 플롯 생성
sm.qqplot(data, line='45')
plt.show()
```

위의 예제에서는 정규 분포에서 생성된 샘플을 이용하여 QQ 플롯을 생성하였습니다.
데이터가 정규 분포를 따를수록 QQ 플롯의 점들이 45도의 대각선에 가깝게 분포됩니다.
`line` 인자를 이용하여 대각선 직선이나 표준화된 데이터의 기대값을 나타내는 직선을 그릴 수 있습니다.
...

```
sm.qqplot(iris['Sepal_Length'], fit=True, line='45')
plt.show()
```

Shapiro-Wilks 검정

```
In [66]: from scipy import stats
```

```
In [1]: '''
scipy.stats 패키지의 shapiro() 함수는 샤피로-윌크 검정(Shapiro-Wilk test)을
수행하는 함수입니다. 이는 데이터가 정규 분포를 따르는지를 검정하는
통계적 방법 중 하나입니다.
이 함수는 주어진 데이터가 정규 분포를 따르는지 여부를 확인할 수 있습니다.

사용법:
scipy.stats.shapiro(x)
```

인자:

- `x`: 정규성을 검정하려는 데이터 배열이나 순차형 데이터.

리턴값:

- `statistic`: 검정 통계량(W 값).
- `p-value`: 귀무 가설(데이터가 정규 분포를 따른다)에 대한 p-값.

특징:

- 귀무 가설(H_0): 데이터가 정규 분포를 따른다.
- 대안 가설(H_1): 데이터가 정규 분포를 따르지 않는다.

검정 통계량(W 값)이나 p-값을 통해 유의수준(α)에 대한 임계치와 비교하여 가설을 채택하거나 기각할 수 있습니다.

주로 p-값이 유의수준보다 작으면 귀무 가설을 기각하고, 데이터가 정규 분포를 따르지 않는다고 결론 내립니다.

...

데이터 생성

```
data = [0.9, 1.2, 1.3, 1.5, 1.6, 1.9, 2.0, 2.1, 2.2]
```

Shapiro-Wilk test 수행

```
statistic, p_value = stats.shapiro(data)
```

```
print("Shapiro-Wilk test statistic:", statistic)
```

```
print("p-value:", p_value)
```

alpha = 0.05 # 유의수준 설정

```
if p_value > alpha:
```

```
    print("귀무 가설을 기각할 증거가 부족하여, 데이터는 정규 분포를 따를 가능성이 있습니다.")
```

```
else:
```

```
    print("귀무 가설을 기각하여, 데이터는 정규 분포를 따르지 않을 가능성이 높습니다.")
```

Shapiro-Wilk test statistic: 0.9515316486358643

p-value: 0.706885039806366

귀무 가설을 기각할 증거가 부족하여, 데이터는 정규 분포를 따를 가능성이 있습니다.

```
In [71]: stats.shapiro(iris['Sepal_Length'])
```

```
Out[71]: ShapiroResult(statistic=0.9760899543762207, pvalue=0.01018026564270258)
```

Kolmogorov-Smirnov 검정

```
In [2]: '''
        scipy.stats 패키지의 kstest() 함수는
        콜모고로프-스미르노프 검정(Kolmogorov-Smirnov test)을 수행하는 함수입니다.
        이 검정은 두 확률 분포가 동일한지를 확인하는데 사용됩니다.
        즉, 주어진 데이터가 특정 분포를 따르는지를 검정하는 통계적 방법 중 하나입니다.

        사용법:
        scipy.stats.kstest(rvs, cdf, args=(), N=20, alternative='two-sided',
                           mode='approx')

        인자:
        - `rvs`: 검정할 데이터 배열.
        - `cdf`: 누적 분포 함수(Cumulative Distribution Function)를 나타내는 문자열,
          또는 함수. 만약 문자열인 경우, 'norm'은 정규 분포를,
          'uniform'은 균일 분포를 의미합니다.
        - `args`: 추가 인수를 포함한 튜플로, CDF에 전달할 매개변수.
        - `N`: 시뮬레이션에 사용할 샘플 크기.
        - `alternative`: 대안 가설을 설정하는데 사용되는 문자열.
          'two-sided'는 양측 검정을,
          'less'는 단측 검정(검정 통계량이 CDF보다 작은 경우),
          'greater'는 단측 검정(검정 통계량이 CDF보다 큰 경우)을 의미합니다.
        - `mode`: 근사 모드를 설정하는데 사용되는 문자열.
          'approx'는 근사 계산을 사용하고, 'asympt'는 비근사적인 계산을 사용합니다.

        리턴값:
        - `statistic`: 검정 통계량(K-S 검정 통계량).
        - `p-value`: 귀무 가설(두 분포가 동일하다)에 대한 p-값.

        특징:
        - 귀무 가설( $H_0$ ): 두 분포는 동일하다.
        - 대안 가설( $H_1$ ): 두 분포는 동일하지 않다.

        검정 통계량(K-S 검정 통계량)과 p-값을 통해 유의수준(alpha)에 대한
        임계치와 비교하여 가설을 채택하거나 기각할 수 있습니다.
        주로 p-값이 유의수준보다 작으면 귀무 가설을 기각하고,
```

두 분포가 동일하지 않다고 결론 내립니다.

`kstest()` 함수를 사용하여 데이터의 분포와 비교할 분포를 통계적으로 검정할 수 있습니다.

```
'''

import numpy as np
import scipy.stats as stats

# 데이터 생성
data = np.random.normal(loc=0, scale=1, size=1000)

# Kolmogorov-Smirnov test 수행
statistic, p_value = stats.kstest(data, 'norm')

print("K-S test statistic:", statistic)
print("p-value:", p_value)

alpha = 0.05 # 유의수준 설정
if p_value > alpha:
    print("귀무 가설을 기각할 증거가 부족하여, 두 분포는 동일할 가능성이 있습니다.")
else:
    print("귀무 가설을 기각하여, 두 분포는 동일하지 않을 가능성이 높습니다.")
```

K-S test statistic: 0.025903568290096546

p-value: 0.5050712816564091

귀무 가설을 기각할 증거가 부족하여, 두 분포는 동일할 가능성이 있습니다.

```
In [72]: m1=iris['Sepal_Length'].mean()
s1=iris['Sepal_Length'].std()
x1=(iris['Sepal_Length']-m1)/s1
stats.kstest(rvs=x1, cdf=stats.norm.cdf)
```

```
Out[72]: KstestResult(statistic=0.08865361377316233, pvalue=0.1781373784859196, statistic_location=-0.8976738791967662, statistic_sign=1)
```

```
In [73]: stats.kstest(iris['Sepal_Length'], stats.norm(m1,s1).cdf)
```

```
Out[73]: KstestResult(statistic=0.08865361377316233, pvalue=0.1781373784859196, statistic_location=5.1, statistic_sign=1)
```

```
In [74]: stats.kstest(iris['Sepal_Length'], iris['Sepal_Width'])
```

```
Out[74]: KstestResult(statistic=0.9933333333333333, pvalue=6.399337692587972e-87, statistic_location=4.2, statistic_sign=-1)
```

Box-Cox transformation

```
In [3]: '''
scipy.stats 패키지의 `boxcox()` 함수는 데이터를 정규 분포에 가깝게 변환하는데
사용되는 강력한 도구입니다. 이 함수는 데이터의 정규성을 증가시키고,
분산을 안정화시키는 등의 장점을 제공합니다.
여러 분야에서 데이터 전처리나 분석에 유용하게 활용됩니다.

사용법:
scipy.stats.boxcox(data, lmbda=None, alpha=None)

매개변수:
- `data`: 변환하려는 데이터. 1차원 배열이어야 합니다.
- `lmbda`: 변환 파라미터. 주어진 값이 없으면,
  데이터로부터 최적의 파라미터가 추정됩니다.
  일반적으로 이 값은 변환의 타입을 결정하는데 사용됩니다.
  예를 들어, `lmbda=0`은 로그 변환을 수행하고,
  `lmbda=None`일 경우 최적의 파라미터를 추정합니다.
- `alpha`: 신뢰 구간을 설정하는데 사용되는 신뢰 수준.
  기본값은 `None`으로, 이 경우 신뢰 구간은 계산되지 않습니다.

반환값:
- `boxcox(data)`: 변환된 데이터를 반환합니다.
- `boxcox(data, lmbda)`: 변환된 데이터와 최적의 람다 값을 반환합니다.
- `boxcox(data, lmbda, alpha)`: 변환된 데이터와 람다 값,
  신뢰 구간을 반환합니다.

이 함수의 주요 특징은 다음과 같습니다:
1. 정규성 증가: 데이터를 정규 분포에 가깝게 만듭니다.
   이는 통계적 가정을 충족시키고,
   많은 통계 분석 기법에 적용 가능하도록 돕습니다.
2. 분산 안정화: 분산이 일정하지 않은 데이터의 경우,
   이를 안정화시켜줍니다.
   이는 통계 분석에서 중요한 요소 중 하나입니다.
```

3. 람다 추정: 최적의 변환 파라미터 람다를 자동으로 추정해줍니다.
이는 데이터에 최적화된 변환을 제공하며,
사용자가 수동으로 값을 설정하지 않아도 됩니다.
4. 신뢰 구간: 변환된 데이터에 대한 신뢰 구간을 계산할 수 있습니다.
이는 변환된 데이터의 신뢰성을 추정하는 데 도움이 됩니다.

요약하자면, `scipy.stats`의 boxcox()` 함수는 데이터를 정규 분포에 가깝게 만들어주는 강력한 도구로, 통계 분석에서 데이터 전처리나 분석에 유용하게 활용됩니다.`

이 예제에서는 지수 분포를 따르는 가상의 데이터를 생성하고, `boxcox()` 함수를 사용하여 이 데이터를 변환합니다. 변환된 데이터와 변환 전의 데이터를 히스토그램으로 시각화하여 비교합니다. 변환된 데이터가 정규 분포에 더 가깝게 분포하는 것을 확인할 수 있습니다.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# 원본 데이터 생성
np.random.seed(0)
data = np.random.exponential(scale=2, size=1000) # 지수 분포를 따르는 가상의 데이터 생성

# Box-Cox 변환 수행
transformed_data, lmbda = stats.boxcox(data)

# 변환된 데이터와 람다 값 출력
print("Lambda parameter:", lmbda)

# 시각화를 위한 원본 데이터와 변환된 데이터 히스토그램
plt.figure(figsize=(10, 5))

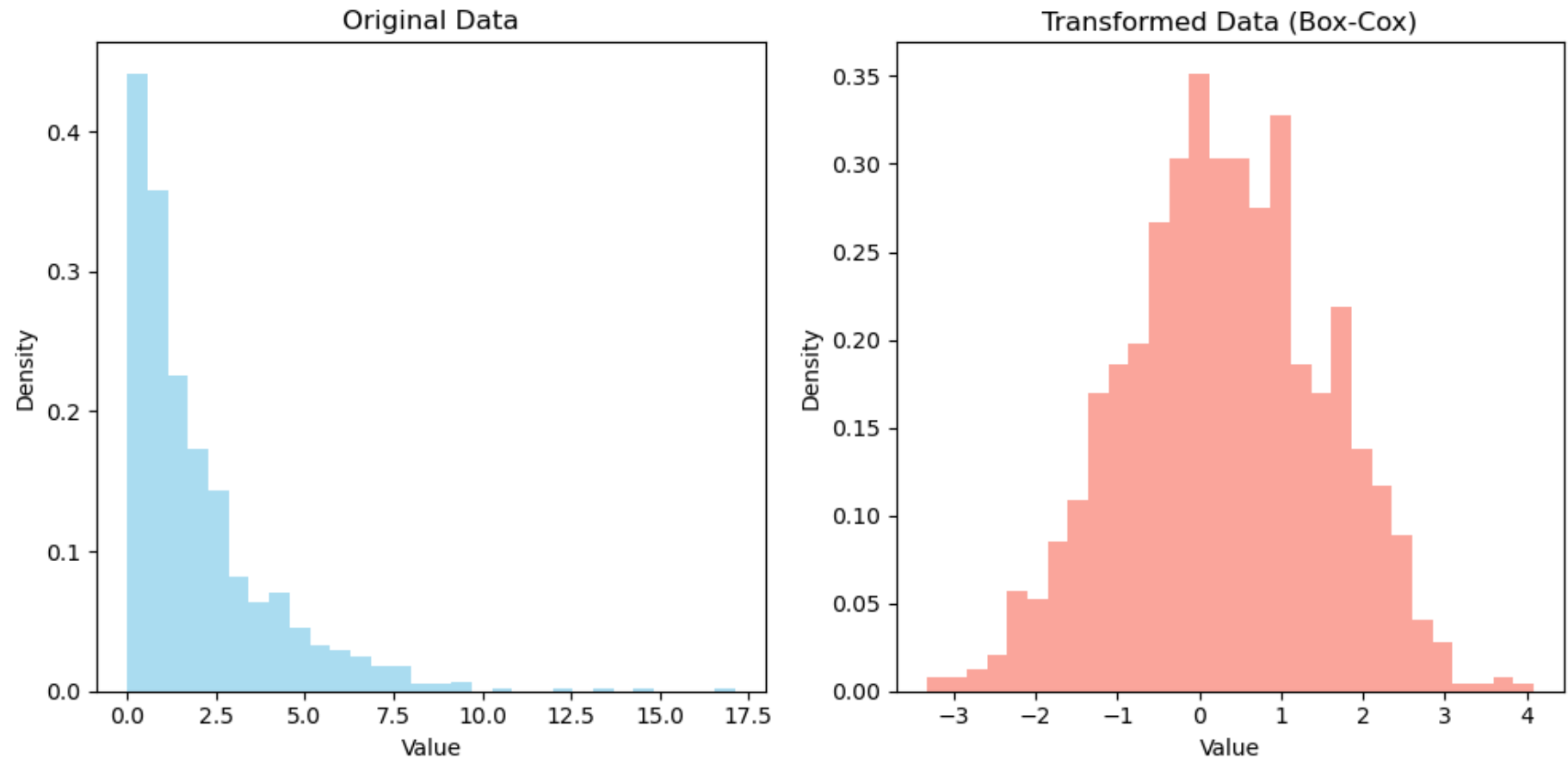
plt.subplot(1, 2, 1)
plt.hist(data, bins=30, density=True, color='skyblue', alpha=0.7)
plt.title('Original Data')
plt.xlabel('Value')
plt.ylabel('Density')

plt.subplot(1, 2, 2)
```

```
plt.hist(transformed_data, bins=30, density=True, color='salmon', alpha=0.7)
plt.title('Transformed Data (Box-Cox)')
plt.xlabel('Value')
plt.ylabel('Density')

plt.tight_layout()
plt.show()
```

Lambda parameter: 0.2420131942268602



In [75]: '''
주어진 코드는 iris 데이터셋에서 'Sepal_Length' 열을 가져와서 Box-Cox 변환을 수행하고,
변환에 사용된 최적의 람다 값을 추정하는 부분입니다.
1. `iris['Sepal\_Length']`: iris 데이터셋에서 'Sepal_Length' 열을 선택합니다.

이는 붓꽃의 꽃받침의 길이를 나타내는 열입니다.

2. ``stats.boxcox(iris['Sepal_Length'])``: 선택된 'Sepal_Length' 열에 대해 Box-Cox 변환을 수행합니다. Box-Cox 변환은 정규 분포로의 변환을 위해 사용됩니다. 이때, 변환된 데이터와 함께 최적의 람다 값이 반환됩니다.
3. ``[1]``: 변환 함수 ``stats.boxcox()``의 반환값은 튜플 형태로 변환된 데이터와 람다 값이 포함되어 있습니다. 여기서 인덱스 1을 사용하여 튜플의 두 번째 요소인 람다 값을 선택합니다.
4. ``lambda_optimal = ...``: 선택된 최적의 람다 값을 ``lambda_optimal`` 변수에 할당합니다.

즉, 이 코드는 iris 데이터셋에서 'Sepal_Length' 열을 선택하고 이를 Box-Cox 변환한 후, 해당 변환에 사용된 최적의 람다 값을 추정하여 ``lambda_optimal`` 변수에 저장합니다. 이 최적의 람다 값은 해당 데이터에 대해 최적의 변환을 제공하는 데 사용될 수 있습니다.

...

``stats.boxcox()`` 함수의 반환값은 Box-Cox 변환된 데이터와 함께 다음과 같은 정보를 포함하는 튜플입니다.

1. ``[0]``: 변환된 데이터
2. ``[1]``: 사용된 람다 값
3. ``[2]``: Box-Cox 변환의 확률성 검정 통계량 (likelihood ratio statistic)

여기서 각각의 요소에 대해 자세히 설명하겠습니다:

1. ``[0]`` - 변환된 데이터: Box-Cox 변환을 거친 후의 데이터입니다. 이는 원본 데이터를 변환하여 얻은 새로운 값들의 배열이며, 정규성을 높이고 분산을 안정화시키는 효과를 가집니다.
2. ``[1]`` - 사용된 람다 값: Box-Cox 변환에 사용된 람다 파라미터입니다. 이 값은 변환의 타입을 결정하며, 일반적으로 자동으로 추정됩니다. 예를 들어, ``lambda = 0``은 로그 변환을 나타냅니다. ``lambda = 1``은 자연로그 변환을 의미하며, ``lambda = None``일 경우 최적의 람다 값이 자동으로 추정됩니다.
3. ``[2]`` - Box-Cox 변환의 확률성 검정 통계량: Box-Cox 변환의 효과를 평가하기 위한 통계량입니다. 이 값은 변환된 데이터가 정규 분포를 따르는지를 검정하는 데 사용됩니다. 일반적으로 이 값이 0에 가까울수록 좋은 변환 효과를 나타냅니다.

이렇게 ``stats.boxcox()`` 함수는 Box-Cox 변환을 수행하고, 변환된 데이터뿐만 아니라 변환에 사용된 람다 값과 변환의 효과를 평가하는 통계량까지 함께 반환하여 사용자에게 제공합니다.


```
lambda_optimal = stats.boxcox(iris['Sepal_Length'])[1]
lambda_optimal
```

Out[75]: -0.14475075607687585

```
In [96]: '''
`np.linspace(-2, 2, 501)`은 NumPy의 linspace 함수를 사용하여
-2부터 2까지의 범위를 501개의 등간격으로 나눈 값을 생성하는 코드입니다.

- `-2`는 시작점을 나타냅니다.
- `2`는 종료점을 나타냅니다.
- `501`은 생성할 숫자의 총 개수를 나타냅니다.

이 코드는 -2부터 2까지의 범위를 501등분하여
해당 등간격에 속하는 값을 반환합니다.
이를 통해 -2와 2 사이의 값들이 등간격으로 총 501개가 생성됩니다.
'''
np.linspace(-2, 2, 501)
```

```

Out[96]: array([-2.    , -1.992, -1.984, -1.976, -1.968, -1.96 , -1.952, -1.944,
-1.936, -1.928, -1.92 , -1.912, -1.904, -1.896, -1.888, -1.88 ,
-1.872, -1.864, -1.856, -1.848, -1.84 , -1.832, -1.824, -1.816,
-1.808, -1.8 , -1.792, -1.784, -1.776, -1.768, -1.76 , -1.752,
-1.744, -1.736, -1.728, -1.72 , -1.712, -1.704, -1.696, -1.688,
-1.68 , -1.672, -1.664, -1.656, -1.648, -1.64 , -1.632, -1.624,
-1.616, -1.608, -1.6 , -1.592, -1.584, -1.576, -1.568, -1.56 ,
-1.552, -1.544, -1.536, -1.528, -1.52 , -1.512, -1.504, -1.496,
-1.488, -1.48 , -1.472, -1.464, -1.456, -1.448, -1.44 , -1.432,
-1.424, -1.416, -1.408, -1.4 , -1.392, -1.384, -1.376, -1.368,
-1.36 , -1.352, -1.344, -1.336, -1.328, -1.32 , -1.312, -1.304,
-1.296, -1.288, -1.28 , -1.272, -1.264, -1.256, -1.248, -1.24 ,
-1.232, -1.224, -1.216, -1.208, -1.2 , -1.192, -1.184, -1.176,
-1.168, -1.16 , -1.152, -1.144, -1.136, -1.128, -1.12 , -1.112,
-1.104, -1.096, -1.088, -1.08 , -1.072, -1.064, -1.056, -1.048,
-1.04 , -1.032, -1.024, -1.016, -1.008, -1.    , -0.992, -0.984,
-0.976, -0.968, -0.96 , -0.952, -0.944, -0.936, -0.928, -0.92 ,
-0.912, -0.904, -0.896, -0.888, -0.88 , -0.872, -0.864, -0.856,
-0.848, -0.84 , -0.832, -0.824, -0.816, -0.808, -0.8 , -0.792,
-0.784, -0.776, -0.768, -0.76 , -0.752, -0.744, -0.736, -0.728,
-0.72 , -0.712, -0.704, -0.696, -0.688, -0.68 , -0.672, -0.664,
-0.656, -0.648, -0.64 , -0.632, -0.624, -0.616, -0.608, -0.6 ,
-0.592, -0.584, -0.576, -0.568, -0.56 , -0.552, -0.544, -0.536,
-0.528, -0.52 , -0.512, -0.504, -0.496, -0.488, -0.48 , -0.472,
-0.464, -0.456, -0.448, -0.44 , -0.432, -0.424, -0.416, -0.408,
-0.4 , -0.392, -0.384, -0.376, -0.368, -0.36 , -0.352, -0.344,
-0.336, -0.328, -0.32 , -0.312, -0.304, -0.296, -0.288, -0.28 ,
-0.272, -0.264, -0.256, -0.248, -0.24 , -0.232, -0.224, -0.216,
-0.208, -0.2 , -0.192, -0.184, -0.176, -0.168, -0.16 , -0.152,
-0.144, -0.136, -0.128, -0.12 , -0.112, -0.104, -0.096, -0.088,
-0.08 , -0.072, -0.064, -0.056, -0.048, -0.04 , -0.032, -0.024,
-0.016, -0.008, 0.    , 0.008, 0.016, 0.024, 0.032, 0.04 ,
0.048, 0.056, 0.064, 0.072, 0.08 , 0.088, 0.096, 0.104,
0.112, 0.12 , 0.128, 0.136, 0.144, 0.152, 0.16 , 0.168,
0.176, 0.184, 0.192, 0.2 , 0.208, 0.216, 0.224, 0.232,
0.24 , 0.248, 0.256, 0.264, 0.272, 0.28 , 0.288, 0.296,
0.304, 0.312, 0.32 , 0.328, 0.336, 0.344, 0.352, 0.36 ,
0.368, 0.376, 0.384, 0.392, 0.4 , 0.408, 0.416, 0.424,
0.432, 0.44 , 0.448, 0.456, 0.464, 0.472, 0.48 , 0.488,
0.496, 0.504, 0.512, 0.52 , 0.528, 0.536, 0.544, 0.552,

```

```

0.56 , 0.568, 0.576, 0.584, 0.592, 0.6 , 0.608, 0.616,
0.624, 0.632, 0.64 , 0.648, 0.656, 0.664, 0.672, 0.68 ,
0.688, 0.696, 0.704, 0.712, 0.72 , 0.728, 0.736, 0.744,
0.752, 0.76 , 0.768, 0.776, 0.784, 0.792, 0.8 , 0.808,
0.816, 0.824, 0.832, 0.84 , 0.848, 0.856, 0.864, 0.872,
0.88 , 0.888, 0.896, 0.904, 0.912, 0.92 , 0.928, 0.936,
0.944, 0.952, 0.96 , 0.968, 0.976, 0.984, 0.992, 1. ,
1.008, 1.016, 1.024, 1.032, 1.04 , 1.048, 1.056, 1.064,
1.072, 1.08 , 1.088, 1.096, 1.104, 1.112, 1.12 , 1.128,
1.136, 1.144, 1.152, 1.16 , 1.168, 1.176, 1.184, 1.192,
1.2 , 1.208, 1.216, 1.224, 1.232, 1.24 , 1.248, 1.256,
1.264, 1.272, 1.28 , 1.288, 1.296, 1.304, 1.312, 1.32 ,
1.328, 1.336, 1.344, 1.352, 1.36 , 1.368, 1.376, 1.384,
1.392, 1.4 , 1.408, 1.416, 1.424, 1.432, 1.44 , 1.448,
1.456, 1.464, 1.472, 1.48 , 1.488, 1.496, 1.504, 1.512,
1.52 , 1.528, 1.536, 1.544, 1.552, 1.56 , 1.568, 1.576,
1.584, 1.592, 1.6 , 1.608, 1.616, 1.624, 1.632, 1.64 ,
1.648, 1.656, 1.664, 1.672, 1.68 , 1.688, 1.696, 1.704,
1.712, 1.72 , 1.728, 1.736, 1.744, 1.752, 1.76 , 1.768,
1.776, 1.784, 1.792, 1.8 , 1.808, 1.816, 1.824, 1.832,
1.84 , 1.848, 1.856, 1.864, 1.872, 1.88 , 1.888, 1.896,
1.904, 1.912, 1.92 , 1.928, 1.936, 1.944, 1.952, 1.96 ,
1.968, 1.976, 1.984, 1.992, 2. ])

```

In [4]:

'''

로그 우도(Log-Likelihood)는 주어진 확률 분포에서
관측된 데이터가 발생할 가능성을 나타내는 함수입니다.

특정 확률 분포에 대한 파라미터가 주어졌을 때,
해당 데이터가 발생할 가능성을 나타내는 값으로,
주로 최대 우도 추정(Maximum Likelihood Estimation, MLE) 과정에서 사용됩니다.

로그 우도는 확률의 곱을 계산하다보면 값이 매우 작아져서
계산이 어려워질 수 있습니다. 따라서 보다 편리한 계산을 위해
보통 로그 변환을 취하여 계산합니다.
이렇게 하면 확률의 곱셈이 로그의 덧셈으로 바뀌어 계산이 간단해지며,
값을 더 쉽게 해석할 수 있습니다.

로그 우도를 최대화하는 것은 주어진 데이터에 대해
가장 적절한 모델의 파라미터를 찾는 것입니다.
이는 최대 우도 추정(MLE)의 기본 아이디어입니다.

최대 우도 추정을 통해 주어진 데이터로부터 모델의
파라미터를 추정할 수 있습니다.
로그 우도가 높을수록 해당 모델이 데이터를 잘 설명한다고 볼 수 있습니다.

따라서 **Box-Cox** 변환의 로그 우도를 계산하는 것은
변환된 데이터가 주어진 람다 값에 대해 얼마나 적합한지를 평가하는데 사용됩니다.
...

...

``scipy.stats`` 패키지의 ``boxcox_1lf()`` 함수는 **Box-Cox** 변환의
로그 우도 함수(**Log-Likelihood Function**) 값을 계산하는 함수입니다.
이 함수는 주어진 데이터와 람다 값에 대한 **Box-Cox** 변환의
로그 우도를 계산하여 반환합니다.
주어진 데이터가 정규 분포를 따르는지를 평가하는 데 사용됩니다.

사용법:
`scipy.stats.boxcox_1lf(lmbda, data)`

매개변수:

- ``lmbda``: **Box-Cox** 변환에 사용될 람다 값입니다.
- ``data``: 변환하려는 데이터입니다.

반환값:

- ``1lf``: 주어진 데이터와 람다 값에 대한 **Box-Cox** 변환의
로그 우도 값입니다.

이 함수의 주요 특징은 다음과 같습니다:

1. 로그 우도 함수 계산: 주어진 데이터와 람다 값에 대한 **Box-Cox** 변환의
로그 우도를 계산합니다. 이를 통해 해당 데이터가 주어진 람다 값에 대해
얼마나 적합한지를 평가할 수 있습니다.
2. 최적화: **Box-Cox** 변환의 로그 우도를 최적화하여 최적의 람다 값을 찾습니다.
이 최적의 람다 값은 데이터에 가장 적합한 변환을 제공합니다.
3. 정규성 평가: 주어진 데이터와 **Box-Cox** 변환의 로그 우도를 통해
데이터가 정규 분포를 따르는지를 평가할 수 있습니다.

아래 예시에서는 주어진 데이터와 람다 값에 대한 **Box-Cox** 변환의
로그 우도를 계산합니다. 계산된 로그 우도는 해당 데이터가
주어진 람다 값에 대해 얼마나 적합한지를 나타냅니다.
...

```
from scipy.stats import boxcox_1lf
```

```
# 주어진 데이터와 람다 값
data = [1, 2, 3, 4, 5]
lmbda = 0.5

# Box-Cox 변환의 로그 우도 계산
llf = boxcox_llf(lmbda, data)
print("Log-Likelihood Function:", llf)
```

Log-Likelihood Function: -1.7005407092322267

```
In [97]: '''
코드는 주어진 람다 값 범위에 대해 Box-Cox 변환의
로그 우도를 계산하는 과정을 보여줍니다.
이 코드를 통해 각 람다 값에 대한 로그 우도를 계산하고
이를 배열에 저장합니다.

여기서 각 변수들에 대한 설명을 해보겠습니다:

- `lambdas`: -2부터 2까지의 범위를 501개의 등간격으로
  나눈 값들을 담은 배열입니다.
  이는 Box-Cox 변환에 사용될 람다 값의 후보들입니다.
- `loglikelihood`: 각 람다 값에 대한 로그 우도를 저장할 배열입니다.
  초기에는 모든 요소가 0으로 초기화됩니다.
- `enumerate(lambdas)`: lambdas 배열의 각 값과 인덱스를 순회합니다.
  람다 값과 해당 인덱스가 순회됩니다.
- `stats.boxcox_llf(1, iris['Sepal_Length'])`: 주어진 람다 값
  `1`에 대해 iris 데이터셋의 'Sepal_Length' 열에 대한
  Box-Cox 변환의 로그 우도를 계산합니다.
- `loglikelihood[i] = ...`: 각 람다 값에 대한 로그 우도를 계산하여
  `loglikelihood` 배열에 저장합니다.
  `i`번째 인덱스에 해당하는 위치에 로그 우도가 저장됩니다.

이 코드를 실행하면 각 람다 값에 대한 로그 우도가 계산되어
`loglikelihood` 배열에 저장됩니다.
이를 통해 주어진 데이터에 대해 어떤 람다 값이
가장 적합한지를 평가할 수 있습니다.
'''

lambdas = np.linspace(-2, 2, 501)
loglikelihood = np.zeros(lambdas.shape, dtype=float)
```

```
for i, l in enumerate(lambdas):
    loglikelihood[i] = stats.boxcox_llf(l, iris['Sepal_Length'])
```

```
In [83]: '''
`loglikelihood.argmax()`는 `loglikelihood` 배열에서
가장 큰 값을 가지는 요소의 인덱스를 반환합니다.
이는 Box-Cox 변환의 로그 우도가 가장 큰 람다 값의 인덱스를 나타냅니다.
'''

loglikelihood.argmax()
```

Out[83]: 232

```
In [85]: '''
`lambdas[232]`는 `lambdas` 배열에서 232번째 인덱스의 값을 나타냅니다.
이는 해당 로그 우도가 최대인 람다 값에 해당합니다.
따라서 이 코드는 Box-Cox 변환의 로그 우도가 가장 큰 람다 값을 찾고,
이에 해당하는 `lambdas` 배열의 값을 반환합니다.

Box-Cox 변환의 로그 우도가 가장 큰 람다 값을 찾는 이유는
해당 람다 값이 주어진 데이터에 가장 적합한 변환을 제공하기 때문입니다.
Box-Cox 변환은 데이터의 분포를 정규 분포에 가깝게 만들어주는
변환 방법 중 하나이며, 이를 통해 데이터의 정규성을 증가시키고
분산을 안정화시킵니다.

로그 우도가 최대가 되는 람다 값은 주어진 데이터에 대해
가장 적합한 변환을 나타냅니다.
즉, 해당 람다 값에 대해 변환된 데이터가
정규 분포를 가장 잘 따른다는 것을 의미합니다.
로그 우도가 높을수록 해당 변환은 데이터를 잘 설명한다고 볼 수 있습니다.

따라서 Box-Cox 변환의 로그 우도가 가장 큰 람다 값을 찾는 것은
데이터에 대해 가장 적합한 변환을 찾는 것과 동일합니다.
이 최적의 람다 값은 데이터를 정규 분포에 가깝게 만들어주고
분석에 적합한 형태로 전처리하는 데 도움을 줍니다.
'''

lambdas[232]
```

Out[85]: -0.14399999999999999

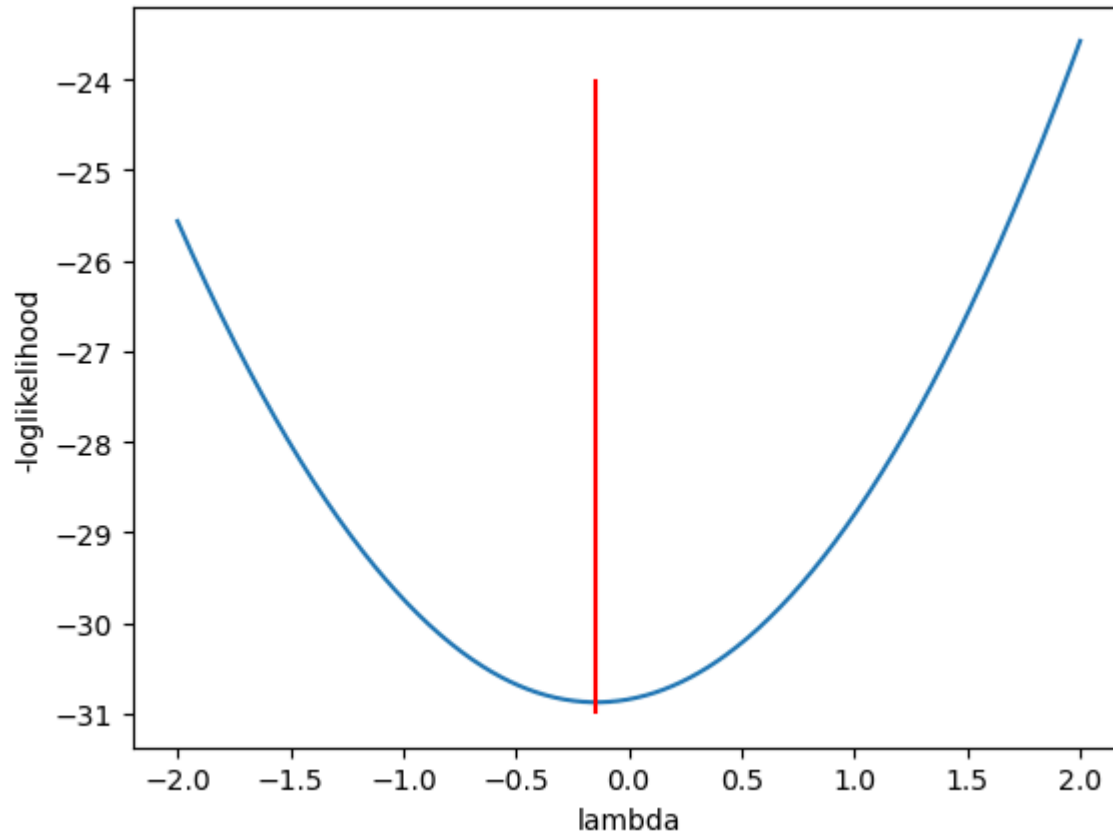
```
In [86]: '''
코드는 주어진 람다 값 범위에 대해 Box-Cox 변환의 로그 우도를 계산하고,
이를 시각화하여 최적의 람다 값을 찾는 과정을 보여줍니다.
시각화를 통해 로그 우도가 가장 큰 람다 값을 쉽게 확인할 수 있습니다.

여기서 사용된 각 변수들에 대해 설명해보겠습니다:

- `fit_boxcox`: 데이터프레임 형태로, 각 람다 값에 대한
  로그 우도를 저장하고 있습니다. 이는 시각화를 위해 사용됩니다.
- `sns.lineplot(data=fit_boxcox, x='lambda', y='-loglikelihood')`:
  `fit_boxcox` 데이터프레임을 사용하여 람다 값에 대한
  로그 우도를 시각화합니다.
  x축은 람다 값이고, y축은 로그 우도 값입니다.
- `plt.vlines(lambda_optimal, ymin=-31, ymax=-24, color='red')`:
  최적의 람다 값에 해당하는 세로선을 그립니다.
  이 때 `lambda_optimal` 변수는 이전에 계산한 최적의 람다 값입니다.
- `plt.show()`: 그래프를 표시합니다.

이를 실행하면 람다 값에 따른 로그 우도를 시각화한 그래프가 나타납니다.
최적의 람다 값에 해당하는 지점을 표시하여 최적의 변환을 쉽게 확인할 수 있습니다.
'''

fit_boxcox = pd.DataFrame({'lambda': lambdas, '-loglikelihood': -loglikelihood})
sns.lineplot(data=fit_boxcox, x='lambda', y='-loglikelihood')
plt.vlines(lambda_optimal, ymin=-31, ymax=-24, color='red')
plt.show()
```



```
In [87]: '''
`stats.shapiro()` 함수는 샤피로-윌크(Shapiro-Wilk) 검정을 수행하여
주어진 데이터가 정규 분포를 따르는지를 검정합니다.
이 함수를 사용하여 로그를 취한 'Sepal_Length' 열의 데이터가
정규 분포를 따르는지를 확인할 수 있습니다.

stats.shapiro(np.log(iris['Sepal_Length']))

위 코드는 'Sepal_Length' 열의 데이터에 자연로그를 취한 후,
해당 데이터가 정규 분포를 따르는지를 검정합니다.
반환되는 값은 샤피로-윌크 검정의 통계량과 p-value로 구성된 튜플입니다.
이를 통해 주어진 데이터가 정규 분포를 따르는지에 대한
유의수준 0.05 하에서의 검정 결과를 확인할 수 있습니다.
'''
```



```
'''
자연로그를 취하는 이유는 주어진 데이터를
정규 분포에 가깝게 만들기 위함입니다.
대부분의 통계적 분석은 정규 분포를 가정하고 있거나,
정규 분포에 가까운 데이터를 필요로 합니다.
그러나 실제 데이터는 종종 정규 분포를 따르지 않을 수 있습니다.
```

```
로그 변환은 데이터의 분포를 정규 분포에 가깝게 만들어주는
일반적인 방법 중 하나입니다.
특히 양수 데이터의 경우에는 로그 변환을 취하면
변환된 데이터가 보다 정규성을 띄는 경우가 많습니다.
이는 로그 변환을 취하면 데이터의 스케일이 줄어들어서
극단값(outlier)의 영향을 줄이고,
데이터의 분포를 정규 분포에 가깝게 만들어주기 때문입니다.
```

```
따라서 'Sepal_Length' 열의 데이터에 자연로그를 취하는 것은
해당 데이터를 정규 분포에 가깝게 만들어주어
통계적 분석에 적합하도록 전처리하는 것입니다.
'''
```

```
stats.shapiro(np.log(iris['Sepal_Length']))
```

Out[87]: ShapiroResult(statistic=0.982528805732727, pvalue=0.05387822538614273)

```
In [91]: '''
이 코드는 주어진 데이터에 대해 콜모고로프-스미르노프 검정
(Kolmogorov-Smirnov test)을 수행하여 해당 데이터가
정규 분포를 따르는지를 검정합니다.
이 때, 자연로그를 취한 'Sepal_Length' 열의
데이터가 정규 분포를 따르는지를 검정하고자 합니다.

먼저 자연로그를 취한 'Sepal_Length' 열의 데이터에 대해
평균과 표준편차를 계산합니다.
그리고 `stats.norm(m2, s2)`를 사용하여
해당 평균과 표준편차를 갖는 정규 분포를 생성합니다.
이후 `stats.kstest()` 함수를 사용하여 콜모고로프-스미르노프 검정을 수행합니다.

이 코드는 주어진 데이터가 정규 분포를 따르는지를 검정하여
p-value를 반환합니다. p-value가 일반적으로 0.05보다 크면
```

해당 데이터가 정규 분포를 따른다는 귀무가설을 기각할 수 없으며,
정규 분포를 따른다고 할 수 있습니다.
'''

```
m2=np.log(iris['Sepal_Length']).mean()  
s2=np.log(iris['Sepal_Length']).std()  
stats.kstest(np.log(iris['Sepal_Length']), stats.norm(m2,s2).cdf)
```

Out[91]: KstestResult(statistic=0.08753842709906018, pvalue=0.18909147691675066, statistic_location=1.62924053973028, statistic_sign=1)

In []: